

根据课件资料，这两种调度的核心区别在于批处理的粒度以及对GPU算力的利用效率。

我们可以用一个非常直观的例子来对比这两种机制：

1. 请求级调度 (Request Scheduling / 传统批处理)

- 运行机制：这种早期模式要求整批请求必须同步开始、同步结束。系统必须等待这一批请求中最慢的那一个全部生成完毕，才能释放资源并接入下一批新请求。
- 举例说明：假设 GPU 同时接收了 3 个用户的并发请求打包成一个 Batch：[请求A, 请求B, 请求C]。
 - 请求A 很短，只需要输出 10 个 Token。
 - 请求B 中等，需要输出 50 个 Token。
 - 请求C 很长，需要输出 100 个 Token。
- 痛点：当处理到第 10 步时，请求A 已经完成了，但它占用的计算“坑位”无法释放，只能闲置干等。直到最长的请求C 跑完 100 个 Token，这一批次才算正式结束，系统才能接入后面的 [请求D, 请求E]。面对 LLM 长短不一的变长输出，这会造成极其巨大的 GPU 算力浪费。

2. Token级调度 (Continuous Batching / 连续批处理)

- 运行机制：这是现代大模型推理引擎（如 vLLM）的核心创新。它将调度的颗粒度细化到了 **Token**（字）级别。用户的请求在同一个推理步骤中进行动态拼接，一旦某个请求完成，立刻将新请求插入空位共享计算，无需等待其他人。
- 举例说明：同样是 [请求A, 请求B, 请求C] 一起开始：
 - 迭代 **1-10**：A、B、C 同步生成各自的 Token。
 - 第 **10** 步：请求A 完成。系统立刻把排队的新请求 [请求D] 插入到 A 空出来的坑位里。
 - 第 **11** 步：此时 GPU 正在并行处理：[请求B的第11步, 请求C的第11步, 新请求D的第1步]。
 - 第 **50** 步：请求B 完成，系统又立刻把 [请求E] 插入进来接替 B 的位置。
- 优势：通过这种动态“插队”机制，GPU 的并行计算槽位几乎永远是满载状态，消除了木桶效应中的短板等待时间，从而大幅提升了整体吞吐量和 GPU 的利用率。

需要我为你进一步解释这种 Continuous Batching (连续批处理) 是如何在 vLLM 框架中具体落地并配合 PagedAttention 发挥最大性能的吗？